Step 1: European MC price

We can price European options with:

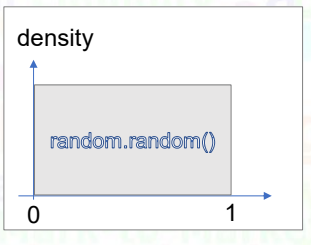
1. Repeat N times:
   1. Draw a random number between 0 and 1
   2. Convert it to a normal draw
   3. Convert it to a Brownian draw
   4. Deduce the value of the underlying on T
   5. Calculate the value of the option on T
   6. Discount to today, this is one result
2. Average the N results: this is the price

python

Random draws in Python

- import random
- Get one random draw in [0, 1) with a uniform density

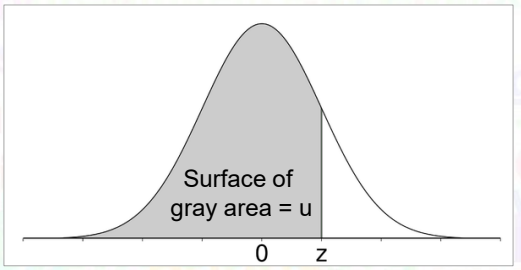
```
import random

[...]
uniformDraw: float = random.random()
```



- Warning: the random number may be equal to 0 (not to 1)!

Normal draw (1)

- The inverse cumulated normal distribution converts a number in (0, 1) into a standard normal draw:
  - N(z): real number z → u = N(z) in (0, 1)
  - $N^{-1}(u)$: u in (0, 1) → z = $N^{-1}(u)$ real number



Normal draw (2)

- from scipy.stats import norm
- Use the method norm.ppf(u)

```
from scipy.stats import norm
[...]
   z: float = norm.ppf(u)
```

- Warning: ppf(0) returns -inf

python

Reset pseudo-random seed

- To update the seed in Python: random.seed(value)
- value is an int
- Example:

```
random.seed(1)
print(random.rand())
random.seed(1)
print(random.rand())
```

⇒

```
0.417022004702574
0.417022004702574
```

python

random.normal()

- random.normal() is different from norm.ppf(random.rand())
- After random.seed(1):

```
uniform: random.rand()
[0.417022 0.720324 0.000114 0.302333 0.146756]
normal: random.normal()
[1.624345 -0.611756 -0.528172 -1.072969 0.865408]
normal from uniform: norm.ppf(random.rand())
[-0.209518 0.583806 -3.684948 -0.517704 -1.050449]
```

- I trust more the uniform draw in random

Generator object

- random.rand(), random.normal(), random.seed() have limitations that are removed by instantiating an object
- Solution: instantiate a random **Generator** object:

```
if seed != 0:
    self._gen: random.Generator = random.default_rng(seed)

[…]

uniform_draw = self._gen.uniform(0, 1)
```

- Please note: RandomState is legacy (link)

Antithetic draws (1)

- A variance-reduction technique consists in using W_t and $-W_t$ to converge faster
- It is efficient if the variance of $\frac{1}{2}\left(f(W_t) + f(-W_t)\right)$ is lower than the variance of $\frac{1}{2}\left(f(W_t) + f(W'_t)\right)$, with W'_t a Brownian independent to W_t
- It improves the convergence if *f(x)* is nearly linear:
  - $\frac{1}{2}(a\,W_t + b + a\,(-W_t) + b) = b$
- It doesn't if *f(x)* is quadratic:
  - $\frac{1}{2}(a\,W_t^2 + a\,(-W_t)^2) = a\,W_t^2$

Antithetic draws (2)

- Example that works well: ITM option with volatility = 3% (*f(x)* nearly linear)
  - Strike ≈ 0: Call price = discounted forward price
  - Std dev without antithetic draws = 0.157 €
  - Std dev with antithetic draws = 0.0065 €

=STDEV.S(B64:Z64)

	A	B	C	D	E	
55						
56	Pricing date	14-Feb-25	14-Feb-25	14-Feb-25	14-Feb-25	14-
57	Nb steps	1	1	1	1	
58	Nb draws	1,000	1,000	1,000	1,000	
59	Am. exerc. frontier	Quadratic	Quadratic	Quadratic	Quadratic	Qu
60	Seed	1	2	3	4	
61						
62		100.0842	100.0784	99.8696	100.2784	
63	0.1566	100.0830	100.0736	99.8658	100.2793	
64	0.0065	99.9886	99.9849	99.9861	99.9910	1
65	-96%					

Antithetic draws (3)

- Less favorable example: ATM option with volatility = 30% (*f(x)* very convex)
  - Std dev without antithetic draws = 1.42 €
  - Std dev with antithetic draws = 0.89 € (-37%)

=STDEV.S(B63:Z63)

	A	B	C	D	E	
55						
56	Pricing date	14-Feb-25	14-Feb-25	14-Feb-25	14-Feb-25	14-
57	Nb steps	1	1	1	1	
58	Nb draws	1,000	1,000	1,000	1,000	
59	Am. exerc. frontier	Quadratic	Quadratic	Quadratic	Quadratic	Qu
60	Seed	1	2	3	4	
61						
62		21.3029	20.9472	19.7562	22.3086	
63	1.4159	21.4083	20.8132	19.0899	22.1823	
64	0.8948	20.3885	19.9178	19.9583	20.6562	
65	-37%					

Scalar vs. vector code

- "Scalar" code: manipulates floating numbers, not vectors
- Scalar is opposed to vector and matrices

Assignment #1:

1. Implement 1 step MC pricing with scalars
2. Implement 1 step MC pricing with vectors

Vectorial MC implementation

- Start by drawing a matrix of Brownian motions:

```
unif_draws: np.ndarray(float) =  ran.uniform(0, 1, (nb_wt, nb_steps))
```

- Convert the uniform draws into scaled normal moves with norm.ppf
- Cumulate them to get a series of Brownian motions with cumsum
- Simulate a series of stock prices with Wt
- Evaluate the regression and the exercise decision for all paths
- Evaluate the option value by path and sum them